

# ISyE6669 HW 5

Spring 2025

1. Compute the gradient  $\nabla f(x)$  and Hessian  $\nabla^2 f(x)$  of the function

$$f(x_1, x_2) = (2x_1 + x_2^2 - 4)^2 + (x_1^2 + x_2 - 8)^2.$$

**Solution:**

$$\nabla f(x) = \begin{bmatrix} 4x_1^3 + 4x_2^2 + 4x_1x_2 - 24x_1 - 16 \\ 4x_2^3 + 2x_1^2 + 8x_1x_2 - 14x_2 - 16 \end{bmatrix}$$

$$\nabla^2 f(x) = \begin{bmatrix} 12x_1^2 + 4x_2 - 24 & 8x_2 + 4x_1 \\ 8x_2 + 4x_1 & 12x_2^2 + 8x_1 - 14 \end{bmatrix}$$

2. Implement the Newton's Method with line search given in Algorithm 1.

---

**Algorithm 1** Newton's Method with Line Search

---

Start with  $x^0$ . Set  $k = 0$ ,  $\epsilon = 10^{-4}$ .  
Set  $d^0 \leftarrow -(\nabla^2 f(x^0))^{-1} \nabla f(x^0)$   
**while**  $\|\nabla f(x^k)\| > \epsilon$  **do**  
    Choose  $\bar{\alpha} > 0, \rho \in (0, 1), c \in (0, 1)$ ; Set  $\alpha^k \leftarrow \bar{\alpha}$   
    **while**  $f(x^k + \alpha^k d^k) > f(x^k) + c\alpha^k \nabla f(x^k)^\top d^k$  **do**  
         $\alpha^k \leftarrow \rho \alpha^k$   
    **end while**  
     $x^{k+1} \leftarrow x^k + \alpha^k d^k$   
     $d^{k+1} \leftarrow -(\nabla^2 f(x^{k+1}))^{-1} \nabla f(x^{k+1})$   
     $k \leftarrow k + 1$   
**end while**

---

Use the Newton's Method to minimize function given in Problem 1. Set the initial stepsize  $\bar{\alpha} = 1$ . Select your own choice of  $\rho \in (0, 1), c \in (0, 1)$ . First run the algorithm from the initial point  $x^0 = (-3, -3)^\top$ , and then try starting point  $x^0 = (-2, -2)^\top$ . For each starting point, print out the step length  $\alpha^k$  used by the algorithm as well as the point  $x^k$  for *every* step  $k$ .

You should observe that Newton's Method converges very fast. Which starting point is better choice? Why?

**Solution:**

**Note:** Answers may vary depending on what  $\rho$  and  $c$  values were used.

Using initial point  $x^0 = (-3, -3)^\top$ , and  $\rho = 0.9$  and  $c = 0.1$ , algorithm converges in 4 iterations and we get the following output from each iteration:

$$k = 0$$

$$x_0 = [-3 \quad -3]$$

$$\alpha_0 = 1$$

$$k = 1$$

$$x_1 = [-3.4508547 \quad -3.34615385]$$

$$\alpha_1 = 1$$

$$k = 2$$

$$x_2 = [-3.36179997 \quad -3.27674837]$$

$$\alpha_2 = 1$$

$$k = 3$$

$$x_3 = [-3.35759836 \quad -3.27341217]$$

$$\alpha_3 = 1$$

$$k = 4$$

$$x_4 = [-3.35758912 \quad -3.27340469]$$

Using initial point  $x^0 = (-2, -2)^\top$ , algorithm converges in 6 iterations and we get the following output from in each iteration:

$$k = 0$$

$$x_0 = [-2 \quad -2]$$

$$\alpha_0 = 1$$

$$k = 1$$

$$x_1 = [1.66666667 \quad 1.77777778]$$

$$\alpha_1 = 0.34867844010000015$$

$$k = 2$$

$$x_2 = [3.07126904 \quad 0.88881759]$$

$$\alpha_2 = 1$$

$$k = 3$$

$$x_3 = [2.72426677 \quad 0.47292701]$$

$$\alpha_3 = 1$$

$$k = 4$$

$$x_4 = [2.69401402 \quad 0.23354192]$$

$$\alpha_4 = 1$$

$$k = 5$$

$$x_5 = [2.69896677 \quad 0.18568618]$$

$$\alpha_5 = 1$$

$$k = 6$$

$$x_6 = [2.69890794 \quad 0.18526019]$$

Note that Newton's method converges very fast in both cases.

**Note:** Any reasonable comparison of starting points should be accepted.

Here is the Python code that generates the above output:

```

10pt
1 import numpy as np
2
3 x0 = np.array([-2, -2])
4 eps = 10**-4

```

```

5 alphabar = 1
6 c = 0.1
7 rho = 0.9
8 f = lambda x1, x2: (2*x1 + x2**2 - 4)**2 + (x1**2 + x2 - 8)**2
9 gradf = lambda x1, x2: np.array([[4*x1**3 + 4*x2**2 + 4*x1*x2 -
    24*x1 - 16],
10                                     [4*x2**3 + 2*x1**2 + 8 * x1*x2 -
    14*x2 - 16]
11                                     ], dtype=np.float64)
12 hessf = lambda x1, x2: np.array([[12*x1**2 + 4*x2 - 24, 8*x2 + 4*
    x1],
13                                     [4*x1 + 8*x2, 12*x2**2 + 8*x1 -
    14]], dtype=np.float64)
14 d = -np.linalg.inv(hessf(x0[0], x0[1])) @ gradf(x0[0], x0[1])
15
16 x = x0.copy()
17 iter = 0
18 while np.linalg.norm(gradf(x[0], x[1])) > eps:
19     alpha = alphabar
20     z = x + (alphabar * d).reshape(-1,)
21     while f(z[0], z[1]) > f(x[0], x[1]) + (c * alpha * gradf(x[0],
    x[1]).T @ d)[0][0]:
22         alpha = rho * alpha
23         z = x + (alpha * d).reshape(-1,)
24     print(f"iter = {iter}, alpha = {alpha}, x={x}")
25     iter += 1
26     x = x + (alpha * d).reshape(-1,)
27     d = -np.linalg.inv(hessf(x[0], x[1])) @ gradf(x[0], x[1])
28 print(f"iter = {iter}, x={x}")

```

3. Figure 1 below illustrates the water network of Newville. The lines are water pipelines numbered from 1 through 11. The arrows on the lines are possible direction(s) of flow of water in these pipelines. The circles are water sources numbered A, B, C. The squares are houses D, E, F, G, H, I. The maximum possible capacity of the water sources are (the sources can operate at less than the maximum capacity): A: 100 Units, B: 100 Units, C: 120 Units. Demands of water in the houses are: D: 50 Units, E: 60 Units, F: 40 Units, G: 30 Units, H: 70 Units, I: 40 Units. Since the houses are at different elevation and the pipes are of different diameter, the cost of transporting water is different in the different pipes. These costs per unit of water are: Pipe 1: \$2, Pipe 2: \$3, Pipe 3: \$4, Pipe 4: \$2, Pipe 5: \$3, Pipe 6: \$2, Pipe 7: \$4, Pipe 8: \$1, Pipe 9: \$2, Pipe 10: \$4, Pipe 11: \$5, Pipe 12: \$3. Formulate an LP to minimize the total cost of transporting water so as to meet the water demands of each house.

#### Solution:

We introduce the following decision variables.  $x_A, x_B, x_C$ , respectively, is the amount of water pumped through water source A, B, C, and  $f_{ij}$  is the flow of water from location  $i$  to location  $j$ ,

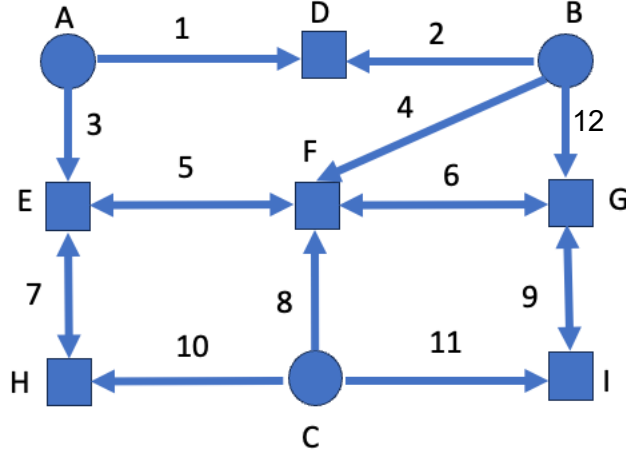


Figure 1: Water Network for Question 2

for arcs  $(i, j)$  in the set of arc  $\mathcal{A} := \{(A, D), (B, D), (A, E), (B, F), (B, G), (E, F), (F, E), (F, G), (G, F), (E, H), (H, E), (C, F), (G, I), (I, G), (C, H), (C, I)\}$ .

$$\begin{aligned}
 \min \quad & 2f_{AD} + 3f_{BD} + 4f_{AE} + 2f_{BF} + 3f_{BG} + 3f_{EF} + 3f_{FE} + 2f_{FG} + 2f_{GF} + 4f_{EH} + 4f_{HE} + \\
 & + f_{CF} + 2f_{GI} + 2f_{IG} + 4f_{CH} + 5f_{CI} \\
 \text{s.t.} \quad & x_A - f_{AD} - f_{AE} = 0 \\
 & x_B - f_{BD} - f_{BF} - f_{BG} = 0 \\
 & x_C - f_{CH} - f_{CF} - f_{CI} = 0 \\
 & f_{AD} + f_{BD} = 50 \\
 & f_{AE} + f_{FE} + f_{HE} - f_{EF} - f_{EH} = 60 \\
 & f_{BF} + f_{CF} + f_{GF} + f_{EF} - f_{FE} - f_{FG} = 40 \\
 & f_{BG} + f_{FG} + f_{IG} - f_{GF} - f_{GI} = 30 \\
 & f_{CH} + f_{EH} - f_{HE} = 70 \\
 & f_{CI} + f_{GI} - f_{IG} = 40 \\
 & x_A \leq 100, \\
 & x_B \leq 100, \\
 & x_C \leq 120, \\
 & x_i \geq 0, \quad \forall i \in \{A, B, C\}, \\
 & f_{ij} \geq 0, \quad \forall (i, j) \in \mathcal{A}.
 \end{aligned}$$

**Note:** Edges  $EF$ ,  $FG$ ,  $GI$  and  $EH$  are bidirectional, meaning that there is flow in both directions. Thus we have two variables for each edge:  $f_{EF}$  and  $f_{FE}$  for  $EF$ ,  $f_{FG}$  and  $f_{GF}$  for  $FG$ ,  $f_{GI}$  and  $f_{IG}$  for  $GI$  and  $f_{EH}$  and  $f_{HE}$  for  $HE$ .

Some of the students assumed that arrows pointing to both directions, indicates that we may

have flow in one direction or the other, but not both at the same time. We will accept that formulation too. Following is the solution in that case.

We introduce the following decision variables.  $x_A, x_B, x_C$ , respectively, is the amount of water pumped through water source A, B, C, and  $f_{ij}$  is the flow of water from location  $i$  to location  $j$ , for arcs  $(i, j)$  in the set of arc  $\mathcal{A} := \{(A, D), (B, D), (A, E), (B, F), (B, G), (C, F), (C, H), (C, I), (E, F), (E, H), (F, G), (G, I)\}$ . And let  $\mathcal{D} = \{(E, F), (E, H), (F, G), (G, I)\} \subset \mathcal{A}$  denotes set of edges for which flow can go in either direction.

$$\begin{aligned}
\min \quad & 2f_{AD} + 3f_{BD} + 4f_{AE} + 2f_{BF} + 3f_{BG} + 3|f_{EF}| + 2|f_{FG}| + 4|f_{EH}| + f_{CF} + 2|f_{GI}| + \\
& + 4f_{CH} + 5f_{CI} \\
\text{s.t.} \quad & x_A - f_{AD} - f_{AE} = 0 \\
& x_B - f_{BD} - f_{BF} - f_{BG} = 0 \\
& x_C - f_{CH} - f_{CF} - f_{CI} = 0 \\
& f_{AD} + f_{BD} = 50 \\
& f_{AE} - f_{EF} - f_{EH} = 60 \\
& f_{BF} + f_{CF} + f_{EF} - f_{FG} = 40 \\
& f_{BG} + f_{FG} - f_{GI} = 30 \\
& f_{CH} + f_{EH} = 70 \\
& f_{CI} + f_{GI} = 40 \\
& x_A \leq 100, \\
& x_B \leq 100, \\
& x_C \leq 120, \\
& x_i \geq 0, \quad \forall i \in \{A, B, C\}, \\
& f_{ij} \geq 0, \quad \forall (i, j) \in \mathcal{A} \setminus \mathcal{D}.
\end{aligned}$$

This is not a linear problem due to two absolute values in the objective function. Hence it will need to be converted into:

$$\begin{aligned}
\min \quad & 2f_{AD} + 3f_{BD} + 4f_{AE} + 2f_{BF} + 3f_{BG} + 3w + 2z + 4s + f_{CF} + 2t + \\
& + 4f_{CH} + 5f_{CI} \\
\text{s.t.} \quad & x_A - f_{AD} - f_{AE} = 0 \\
& x_B - f_{BD} - f_{BF} - f_{BG} = 0 \\
& x_C - f_{CF} - f_{CG} - f_{CH} = 0 \\
& f_{AD} + f_{BD} = 50 \\
& f_{AE} - f_{EF} - f_{EH} = 60 \\
& f_{BF} + f_{CF} + f_{EF} - f_{FG} = 40 \\
& f_{BG} + f_{FG} - f_{GI} = 30 \\
& f_{CH} + f_{EH} = 70 \\
& f_{CI} + f_{GI} = 40 \\
& x_A \leq 100, \\
& x_B \leq 100, \\
& x_C \leq 120, \\
& x_i \geq 0, \quad \forall i \in \{A, B, C\}, \\
& f_{ij} \geq 0, \quad \forall (i, j) \in \mathcal{A} \setminus \mathcal{D} \\
& -w \leq f_{EF} \leq w \\
& -z \leq f_{FG} \leq z \\
& -s \leq f_{EH} \leq s \\
& -t \leq f_{GI} \leq t.
\end{aligned}$$

4. Consider the following electric power network shown in Figure 2. This network is taken from a real-world electric power system. Electricity generators are located at nodes 1, 3, and 5 and producing  $p_1, p_2, p_3$  amounts of electricity, respectively. Electricity loads are located at nodes 2, 4, and 6 and are consuming  $d_1, d_2, d_3$  amounts of electricity, respectively.

The demand is fixed and given as  $d_1 = 120$ ,  $d_2 = 95$ ,  $d_3 = 105$ .

Each generator  $i$ 's production must be within an upper and a lower bound as  $p_i^{\min} \leq p_i \leq p_i^{\max}$ . The bounds are given as  $p_1^{\min} = 20$ ,  $p_1^{\max} = 270$ ,  $p_2^{\min} = 20$ ,  $p_2^{\max} = 250$ ,  $p_3^{\min} = 10$ ,  $p_3^{\max} = 300$ .

The flow limits over lines are given as  $f_{12}^{\max} = 100$ ,  $f_{23}^{\max} = 120$ ,  $f_{34}^{\max} = 50$ ,  $f_{45}^{\max} = 90$ ,  $f_{56}^{\max} = 60$ ,  $f_{61}^{\max} = 50$ .

The line parameters are given as  $B_{12} = 11.6$ ,  $B_{23} = 5.9$ ,  $B_{34} = 13.7$ ,  $B_{45} = 9.8$ ,  $B_{56} = 5.6$ ,  $B_{61} = 10.5$ . The unit generation costs are given as  $c_1 = 10$ ,  $c_2 = 5$ ,  $c_3 = 8$ .

- Formulate the power system scheduling problem using the model discussed in Lecture 2.
- Implement and solve the model using CVXPY. Write down the optimal solution.

- (c) Find the electricity prices for demand at nodes 2, 4, 6. To do this, use the command `constraints[0].dual_value` to find the dual variable of `constraints[0]`. Hint: Recall the electricity price at node  $i$  is the dual variable for the flow conservation constraint at node  $i$ .

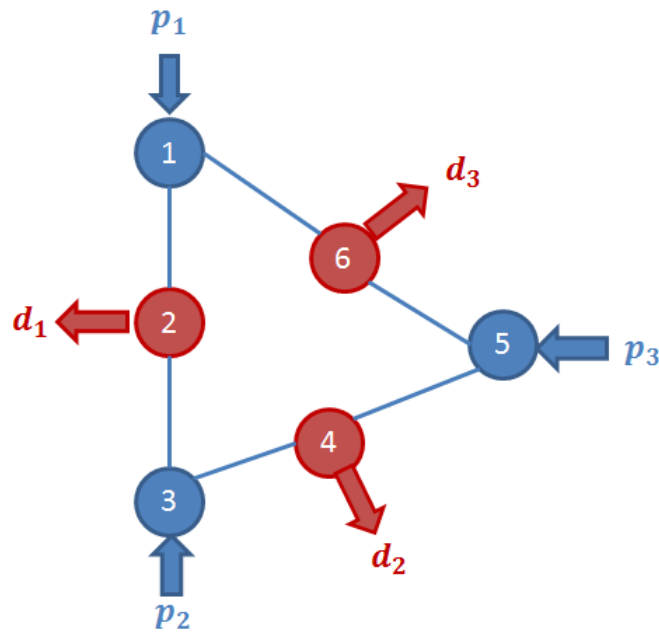


Figure 2: Electric network for Question 3.



(a) The LP can be formulated as follows:

$$\min \quad 10p_1 + 5p_2 + 8p_3$$

s.t.

*Flow Conservation:*

$$f_{12} - f_{61} = p_1,$$

$$f_{34} - f_{23} = p_2,$$

$$f_{56} - f_{45} = p_3,$$

$$f_{23} - f_{12} = -120,$$

$$f_{45} - f_{34} = -95,$$

$$f_{61} - f_{56} = -105,$$

*Constraints linking branch flow and nodal potential:*

$$f_{12} = 11.6(\theta_1 - \theta_2),$$

$$f_{23} = 5.9(\theta_2 - \theta_3),$$

$$f_{34} = 13.7(\theta_3 - \theta_4),$$

$$f_{45} = 9.8(\theta_4 - \theta_5),$$

$$f_{56} = 5.6(\theta_5 - \theta_6),$$

$$f_{61} = 10.5(\theta_6 - \theta_1),$$

*Flow limit constraints:*

$$-100 \leq f_{12} \leq 100,$$

$$-120 \leq f_{23} \leq 120,$$

$$-50 \leq f_{34} \leq 50,$$

$$-90 \leq f_{45} \leq 90,$$

$$-60 \leq f_{56} \leq 60,$$

$$-50 \leq f_{61} \leq 50,$$

*Generation constraints:*

$$20 \leq p_1 \leq 270,$$

$$20 \leq p_2 \leq 250,$$

$$10 \leq p_3 \leq 300.$$

(b) The optimal objective value is 2430.43. The amounts of electricity produced are the following:

$$p_1 = 103.09,$$

$$p_2 = 111.91,$$

$$p_3 = 105.00$$

.

The flows in different lines are following:

$$\begin{aligned}
f_{12} &= 58.09, \\
f_{23} &= -61.91, \\
f_{34} &= 50.00, \\
f_{45} &= -45.00, \\
f_{56} &= 60.00, \\
f_{61} &= -45.00
\end{aligned}$$

(c) The electricity prices are:

$$\begin{aligned}
\text{node 2 : } &8.31, \\
\text{node 4 : } &10.0, \\
\text{node 6 : } &11.86.
\end{aligned}$$

Code used for parts (b) and (c):

```

10pt
1 import cvxpy as cvx
2 import numpy as np
3
4 #inputs provides
5 p_min=np.array([20,20,10])
6 p_max=np.array([270,250,300])
7 f_max=np.array([100,120,50,90,60,50])
8 B=[11.6,5.9,13.7,9.8,5.6,10.5]
9 c=np.array([10,5,8])
10 d=[120,95,105]
11
12 #declaring variables
13 p=cvx.Variable(3)
14 f=cvx.Variable(6)
15 o=cvx.Variable(6)
16
17 #defining objective
18 objective = cvx.Minimize(p@c)
19
20 #defining constraints
21 constraints= [f[0]-f[-1]==p[0],
22               f[2]-f[1]==p[1],
23               f[4]-f[3]==p[2],
24               f[1]-f[0]==-d[0],
25               f[3]-f[2]==-d[1],

```

```

26         f[5] - f[4] == -d[2] ,
27
28         f[0] == B[0] * (o[0] - o[1]) ,
29         f[1] == B[1] * (o[1] - o[2]) ,
30         f[2] == B[2] * (o[2] - o[3]) ,
31         f[3] == B[3] * (o[3] - o[4]) ,
32         f[4] == B[4] * (o[4] - o[5]) ,
33         f[5] == B[5] * (o[5] - o[0]) ,
34
35         -f_max <= f ,
36         f <= f_max ,
37
38         p_min <= p ,
39         p <= p_max ,
40
41     ]
42
43 myprob = cvx.Problem(objective , constraints)
44 myprob.solve()
45
46 #printing outputs
47 print("\\nThe optimal value is" , round(myprob.value,2))
48 print("The amount of electricity produced at 1,3,5 are" ,
49     [round(x,2) for x in p.value])
50 print("The flows in different lines are" ,
51 [round(x,2) for x in f.value])
52 print("The electrierty price at node 2 is" ,
53 round(constraints[3].dual_value,2) )
54 print("The electrierty price at node 4 is" ,
55 round(constraints[4].dual_value,2) )
56 print("The electrierty price at node 6 is" ,
57 round(constraints[5].dual_value,2) )

```